Machine Learning for Combinatorial Optimization: A Literature Review

Abstract

Combinatorial optimization problems are ubiquitous in science, engineering, and industry, yet their NP-hard nature makes them computationally intractable for classical exact algorithms at scale. Recent advances in machine learning have opened new avenues for tackling these challenging problems, offering data-driven approaches that can learn problem-specific heuristics and exploit structural patterns. This literature review synthesizes recent research on machine learning for combinatorial optimization, examining 30 highly relevant papers spanning deep reinforcement learning, graph neural networks, and hybrid methods that integrate learning with traditional optimization techniques. We identify three primary methodological paradigms: end-to-end deep reinforcement learning approaches that learn constructive or improvement heuristics, graph neural network methods that leverage problem structure through unsupervised or physics-inspired training, and hybrid frameworks that combine machine learning with constraint programming, column generation, or branch-and-bound. The review covers applications across diverse problem domains including routing problems, scheduling, graph optimization, and mixed-integer programming. Key findings indicate that machine learning methods can achieve competitive or superior performance compared to classical heuristics while offering better generalization and scalability. However, challenges remain in terms of solution quality guarantees, interpretability, and the computational overhead of training. We conclude by discussing promising future directions including meta-learning, integration with large language models, and the development of unified frameworks for neural combinatorial optimization.

Contents

1 Introduction **3**

2 Background and Theoretical Foundations **3**
2.1 Combinatorial Optimization Problems 3
2.2 Machine Learning Paradigms for Optimization 4
2.3 Neural Network Architectures 4
2.4 Training Paradigms 4

3 Deep Reinforcement Learning Approaches **5**
3.1 Constructive Methods 5
3.2 Improvement and Search Methods 5
3.3 Multi-Objective Optimization 6
3.4 Novel Architectures and Training Strategies 6

4 Graph Neural Network Methods **6**
4.1 Unsupervised and Physics-Inspired Approaches 6
4.2 State-Aware and Relational Methods 7
4.3 Search Space Pruning 7
4.4 Domain-Specific Applications 7

5 Hybrid Approaches: Integrating Learning with Classical Optimization **8**
5.1 Learning-Enhanced Branch-and-Bound . . . 8
5.1.1 Imitation of Strong Branching . . . 8
5.1.2 Graph Neural Network Branching Policies . . . 9
5.1.3 Reinforcement Learning for Branching . . . 9
5.1.4 Online and Adaptive Branching . . . 9
5.1.5 Node Selection, Diving, and Primal Heuristics . . . 10
5.2 Constraint Programming Integration . . . 10
5.3 Column Generation Enhancement . . . 10
5.4 Mixed-Integer Programming Acceleration . . . 11
5.5 Learning Problem Approximations . . . 11

6 Problem Domains and Applications **11**
6.1 Routing Problems . . . 12
6.2 Scheduling Problems . . . 12
6.3 Graph Optimization Problems . . . 12
6.4 Mixed-Integer Programming . . . 13
6.5 Constraint Optimization Problems . . . 13
6.6 Emerging Applications . . . 13

7 Discussion **13**
7.1 Performance and Scalability . . . 13
7.2 Training Requirements and Data Efficiency . . . 14
7.3 Solution Quality and Optimality Guarantees . . . 14
7.4 Interpretability and Insights . . . 14
7.5 Generalization and Robustness . . . 15
7.6 Computational Infrastructure Requirements . . . 15
7.7 Benchmarking and Reproducibility . . . 15

8 Future Directions and Open Challenges **15**
8.1 Unified Frameworks and Architectures . . . 15
8.2 Theoretical Understanding . . . 16
8.3 Large Language Models and Foundation Models . . . 16
8.4 Hybrid and Neuro-Symbolic Approaches . . . 16
8.5 Real-World Deployment and Applications . . . 16
8.6 Multi-Task and Transfer Learning . . . 17
8.7 Explainability and Trust . . . 17
8.8 Benchmarking and Evaluation Standards . . . 17
8.9 Quantum and Neuromorphic Computing . . . 17

9 Conclusion **18**

1 Introduction

Combinatorial optimization problems constitute a fundamental class of computational challenges that arise across numerous domains, from logistics and manufacturing to telecommunications and computational biology. These problems involve finding optimal configurations from a finite but exponentially large set of discrete possibilities, such as determining the shortest tour visiting all cities in the Traveling Salesman Problem (TSP), scheduling jobs on machines to minimize completion time, or partitioning graphs to optimize specific objectives. The NP-hard complexity of most combinatorial optimization problems means that exact algorithms become computationally prohibitive as problem size increases, necessitating the development of heuristic and approximation methods [1].

Traditional approaches to combinatorial optimization have relied on carefully designed heuristics, metaheuristics, and mathematical programming techniques. While these methods have achieved remarkable success, they typically require substantial domain expertise and manual tuning for each problem class. Moreover, classical heuristics often struggle to adapt to new problem variants or exploit problem-specific structure that may be present in real-world instances [2].

The recent surge in machine learning capabilities, particularly in deep learning and reinforcement learning, has catalyzed a paradigm shift in how we approach combinatorial optimization. Machine learning offers the promise of automatically discovering effective solution strategies by learning from data, whether through supervised learning from optimal solutions, reinforcement learning through trial and error, or unsupervised learning that exploits problem structure. These data-driven approaches can potentially generalize across problem instances, adapt to new problem variants, and scale to larger instances than those seen during training [3].

This literature review examines the state-of-the-art in machine learning for combinatorial optimization, synthesizing insights from 30 highly relevant recent papers. We organize the review around three primary methodological paradigms that have emerged in the field. First, deep reinforcement learning approaches train neural networks to construct solutions sequentially or improve existing solutions through learned policies [4], [5], [6]. Second, graph neural network methods leverage the inherent graph structure of many combinatorial problems to learn representations and solution strategies [7], [8], [9]. Third, hybrid approaches integrate machine learning components with classical optimization techniques such as constraint programming, branch-and-bound, and column generation to enhance traditional solvers [10], [11], [12].

The review is structured as follows. Section 2 provides theoretical foundations and contextualizes machine learning within the broader landscape of combinatorial optimization. Sections 3, 4, and 5 examine the three main methodological paradigms in detail. Section 6 surveys applications across different problem domains. Section 7 critically analyzes the current state of the field, including performance comparisons, scalability considerations, and limitations. Section 8 identifies promising future research directions, and Section 9 concludes the review.

2 Background and Theoretical Foundations

2.1 Combinatorial Optimization Problems

Combinatorial optimization problems seek to find an optimal object from a finite set of discrete objects. Formally, a combinatorial optimization problem can be expressed as finding $x^* \in \mathcal{X}$ that minimizes (or maximizes) an objective function $f(x)$, where $\mathcal{X}$ represents a discrete feasible set. The discrete nature distinguishes these problems from continuous optimization and often leads to computational intractability. Many fundamental combinatorial problems, including TSP, graph coloring, maximum cut, and various scheduling problems, are NP-hard, meaning no polynomial-time algorithm is known to solve them optimally in the worst case [1].

Classical solution approaches include exact methods such as branch-and-bound, dynamic programming, and integer programming, which guarantee optimal solutions but may require exponential time. Heuristic methods sacrifice optimality guarantees for computational efficiency, while metaheuristics like genetic algorithms, simulated annealing, and tabu search provide general frameworks applicable across problem classes [13]. Approximation algorithms offer theoretical performance guarantees, ensuring solutions within a bounded factor of the optimum [1].

2.2 Machine Learning Paradigms for Optimization

The integration of machine learning with combinatorial optimization can be categorized into several paradigms based on how learning is employed [1], [3]. In the *end-to-end learning* paradigm, neural networks are trained to directly output solutions or solution components, effectively replacing traditional solvers. This approach is exemplified by sequence-to-sequence models that construct solutions autoregressively [5], [6].

The *learning to optimize* paradigm uses machine learning to enhance components of traditional optimization algorithms. This includes learning branching rules in branch-and-bound [12], learning cutting plane selection strategies, or learning variable ordering heuristics in constraint programming [14], [15]. Such hybrid approaches aim to combine the strengths of both learning-based and classical methods [16].

A third paradigm involves *learning problem structure*, where machine learning models identify patterns or structure in problem instances that can guide solution methods. This includes learning to prune search spaces [17], predicting solution quality or bounds [18], or learning problem approximations [19].

2.3 Neural Network Architectures

Several neural network architectures have proven particularly effective for combinatorial optimization. *Sequence-to-sequence models* with attention mechanisms, originally developed for natural language processing, have been adapted to construct solutions sequentially [6], [20]. The attention mechanism allows the model to focus on relevant parts of the input when making each decision.

Graph neural networks (GNNs) have emerged as a natural fit for combinatorial problems with graph structure [3]. GNNs operate on graph-structured data by iteratively aggregating information from neighboring nodes, allowing them to learn representations that respect graph topology. Variants include graph convolutional networks, graph attention networks, and message-passing neural networks [7], [8], [9].

Pointer networks extend sequence-to-sequence models with a pointing mechanism that selects elements from the input sequence, making them well-suited for problems where the output is a permutation or subset of the input [21], [20]. *Transformer architectures* with self-attention have also been successfully applied, offering improved parallelization and the ability to capture long-range dependencies [21].

2.4 Training Paradigms

Machine learning models for combinatorial optimization can be trained through various paradigms. *Supervised learning* requires labeled training data, typically optimal or high-quality solutions obtained from exact solvers or expert heuristics [14], [12]. While this approach can achieve strong performance, it is limited by the availability and cost of generating training labels, particularly for large instances.

Reinforcement learning (RL) trains agents to make sequential decisions by maximizing cumulative rewards [2]. In the context of combinatorial optimization, the agent learns a policy that

constructs or improves solutions, receiving rewards based on solution quality. RL methods include policy gradient algorithms like REINFORCE [5], [6], actor-critic methods, and Q-learning variants [22], [15]. The advantage of RL is that it requires only a reward signal (typically the objective function value) rather than optimal solutions.

Unsupervised learning approaches train models without labeled solutions, instead using the problem's objective function directly as a loss function [7], [8]. This is particularly appealing as it eliminates the need for training data generation. Physics-inspired approaches formulate the optimization problem as a Hamiltonian and train GNNs to minimize this energy function [8].

Meta-learning aims to learn optimization strategies that generalize across problem instances or even problem classes [5]. By training on a distribution of problem instances, meta-learning approaches can achieve better initialization and faster adaptation to new instances [5].

3 Deep Reinforcement Learning Approaches

Deep reinforcement learning has emerged as one of the most prominent paradigms for neural combinatorial optimization, offering the ability to learn solution construction or improvement strategies without requiring labeled optimal solutions. This section examines the major approaches and innovations in applying deep RL to combinatorial problems.

3.1 Constructive Methods

Constructive RL methods learn to build solutions incrementally by making sequential decisions. These approaches typically model the solution process as a Markov decision process where the agent selects solution components one at a time until a complete solution is formed.

DIMES introduces a differentiable meta-solver that parameterizes solution distributions in a compact continuous space, enabling massively parallel sampling and REINFORCE-based training [5]. Unlike traditional autoregressive decoding methods that suffer from sequential bottlenecks, DIMES achieves scalability to graphs with tens of thousands of nodes by avoiding costly decoding steps. The meta-learning framework provides effective parameter initialization for fine-tuning on specific instances, demonstrating strong performance on TSP and maximal independent set problems [5].

For the Covering Salesman Problem, a deep RL approach using an encoder-decoder architecture with multi-head attention has been developed [6]. The key innovation is a dynamic embedding based on a guidance vector that captures the changing covering states of cities during solution construction. This model runs more than 20 times faster than traditional heuristic solvers while maintaining high solution quality and generalizing across different problem variants [6].

The application of deep RL to the one-dimensional cutting stock problem demonstrates the versatility of these methods beyond routing problems [20]. Using a pointer network with LSTM units as the policy network, the approach models cutting sequence selection as a Markov decision process trained via Monte Carlo reinforcement learning. The method achieves high-efficiency approximate solutions on 82 instances across three datasets in under one second per instance [20].

3.2 Improvement and Search Methods

While constructive methods build solutions from scratch, improvement methods learn to refine existing solutions through local search or perturbation strategies.

Exploratory Combinatorial Optimization with Deep Q-learning (ECO-DQN) learns to continuously improve solutions by exploring the solution space at test time [22]. A key advantage is that ECO-DQN can start from any arbitrary configuration, enabling combination with other search methods such as random search. This approach achieves state-of-the-art RL performance on the maximum cut problem [22].

Efficient active search strategies address a critical limitation of standard RL approaches: the inability to adapt model parameters to specific test instances [4]. Rather than updating all model weights during test-time adaptation, which is computationally expensive, efficient active search updates only a subset of parameters. This approach significantly improves search performance and even surpasses the well-known LKH3 heuristic solver on the capacitated vehicle routing problem. Importantly, efficient active search enables learned models to solve instances much larger than those seen during training [4].

3.3 Multi-Objective Optimization

Extending neural combinatorial optimization to multi-objective settings presents additional challenges, as solutions must balance multiple competing objectives. Pareto Set Learning addresses this by training a preference-conditioned model that can approximate the entire Pareto front [23]. The approach uses an attention-based encoder-decoder architecture where decoder parameters are conditioned on trade-off preferences via MLP hypernetworks. A multi-objective REINFORCE algorithm trains all preferences simultaneously, achieving superior solution quality, speed, and model efficiency across multi-objective TSP, vehicle routing, and knapsack problems. An active adaptation method further enhances performance on out-of-distribution instances [23].

3.4 Novel Architectures and Training Strategies

Recent work has explored alternative neural architectures and training paradigms beyond standard policy gradient methods. IC/DC introduces the first diffusion model framework for combinatorial optimization, trained in a self-supervised manner without costly supervision [24]. The iterative training approach alternates between a cloning step and an improvement step using reinforcement learning. The architecture combines dual graph attentional layers for problem encoding with an anisotropic graph neural network for denoising. This feasibility-enforced generation process achieves state-of-the-art deep learning performance on parallel machine scheduling and asymmetric TSP [24].

The RL4CO benchmark provides a unified, modular framework for evaluating reinforcement learning approaches to combinatorial optimization [25]. By decoupling autoregressive solvers and training routines, RL4CO enables systematic comparison of methods across diverse problems including TSP, vehicle routing variants, and scheduling problems. Benchmark results reveal that simple augmentation strategies can outperform more complex sampling methods, and some recent state-of-the-art methods fall behind their predecessors when evaluated on metrics beyond single-instance performance [25].

4 Graph Neural Network Methods

Graph neural networks have emerged as a powerful tool for combinatorial optimization, particularly for problems with inherent graph structure. GNNs can learn representations that respect graph topology and exploit structural patterns, making them well-suited for graph-based optimization problems.

4.1 Unsupervised and Physics-Inspired Approaches

A significant advantage of GNN-based methods is their ability to train in an unsupervised manner, eliminating the need for labeled training data. Physics-inspired GNN approaches formulate combinatorial problems as energy minimization tasks, connecting spin Hamiltonians and QUBO/PUBO formalisms to differentiable loss functions [8]. Using graph convolutional networks with custom loss functions derived from relaxed problem Hamiltonians, these methods train via gradient descent without pre-labeled datasets. Deterministic projection heuristics map

soft probabilities back to binary variables. This approach performs on par with or outperforms existing solvers while scaling to problems with millions of variables across maximum cut, maximum independent set, minimum vertex cover, and other graph problems [8].

Chaotic Graph Backpropagation (CGBP) introduces a brain-inspired training algorithm that leverages chaotic dynamics to prevent GNNs from falling into local minima [7]. By incorporating a local loss function inspired by neural dynamics, CGBP achieves global ergodicity and pseudo-randomness that enhances exploration during training. The method achieves high-precision solutions with linear time complexity $O(n)$ for large-scale problems including maximum independent set, maximum cut, and graph coloring. Notably, CGBP functions as a universal plug-in unit that can enhance existing GNN architectures [7].

4.2 State-Aware and Relational Methods

Capturing the evolving state of partial solutions during the optimization process is crucial for effective learning. The State-Aware Multi-relation Aggregation (SAMA) model addresses this by accurately capturing information in both graph structure and partial solutions at intermediate steps [9]. SAMA demonstrates superior performance over alternative algorithmic and learning-based models on both artificially generated graphs and real-world application graphs [9].

For bilevel optimization problems, GNNs can model complex hierarchical structures. A deep learning approach for the Bilevel Knapsack Problem models the problem as a tripartite graph and applies Principal Neighborhood Aggregation Neural Networks combined with supervised learning [26]. The GNN predicts the leader's binary decision variables, transforming the hierarchical problem into a single-level knapsack problem solved via CPLEX. This approach achieves feasible solutions approximately 500 times faster than exact algorithms with only a 1.7% optimality gap and demonstrates strong generalization across problem sizes [26].

4.3 Search Space Pruning

Machine learning can identify and prune elements unlikely to appear in optimal solutions, dramatically reducing search space size. An interpretable ML framework uses supervised learning with probabilistic classification models based on local features to repeatedly prune non-optimal elements [17]. Employing gradient boosted trees and logistic regression, the approach demonstrates multi-stage sparsification on maximum clique enumeration, pruning around 99% of nodes in sparse graphs while retaining maximum cliques. The introduction of local chromatic density and chi-squared statistical features provides deeper problem insights and leads to the novel ALTHEA clique-finding heuristic for dense graphs [17].

Logistic regression combined with simulated annealing provides another approach to search space pruning for binary combinatorial problems [27]. The logistic regression model estimates the probability of elements belonging to optimal solutions, guiding the simulated annealing search toward promising regions. Experiments on the multidimensional knapsack problem and subset sum problem demonstrate that this hybrid approach enhances search quality [27].

4.4 Domain-Specific Applications

GNN methods have been successfully applied to specialized problem domains. For the dominating set problem and its variants, a machine learning framework using neural networks frames the problem as a constraint satisfaction problem, integrating learning directly into the discrete optimization process [28].

The integration of metaheuristics with large language models represents an emerging direction [29]. By employing LLMs as pattern recognition engines to analyze node metrics, probabilities computed from LLM outputs can guide biased random key genetic algorithms.

This integrated optimization approach demonstrates improved performance over state-of-the-art deep learning methods and pure metaheuristics on the multi-hop influence maximization problem [29].

5 Hybrid Approaches: Integrating Learning with Classical Optimization

Hybrid approaches that combine machine learning with classical optimization techniques have emerged as a particularly promising direction, leveraging the complementary strengths of both paradigms. These methods integrate learning components into established optimization frameworks such as constraint programming, branch-and-bound, and column generation.

5.1 Learning-Enhanced Branch-and-Bound

Branch-and-bound (B&B) is the algorithmic backbone of modern mixed-integer programming (MIP) solvers such as CPLEX and Gurobi. Its efficiency depends critically on several interleaved decisions: which variable to branch on, which node to explore next, when to apply cutting planes, and how to find feasible solutions early via primal heuristics. Machine learning has been applied to each of these decisions, with branching variable selection receiving the most sustained attention.

5.1.1 Imitation of Strong Branching

Strong branching (SB) selects the variable whose branching produces the largest dual bound improvement among all candidates. Despite its powerful guidance, SB is prohibitively expensive because it requires solving two LP relaxations per candidate variable at each node. Machine learning addresses this by training a fast surrogate that imitates SB decisions without paying its full computational cost.

The seminal work of Khalil et al. [30] introduced an on-the-fly imitation framework: SB labels are collected during the early phase of a single solve, and an instance-specific ranking model is fitted online and subsequently used to guide the remaining search. This approach yields significantly smaller B&B trees than classical heuristics such as pseudocost branching and achieves competitive performance with commercial solvers on standard benchmarks [30]. A key insight is that collecting labels and fitting the model during the same solve exploits instance-specific structure unavailable to pre-trained models, though it introduces SB overhead in the labelling phase.

Gasse et al. [31] replaced hand-crafted features with a bipartite graph convolutional neural network (GCNN) that represents MIP instances as variable–constraint graphs and is trained offline via imitation of SB across a corpus of problem instances. At inference, the GCNN produces branching scores in a single forward pass, achieving superior performance over prior ML methods and solver heuristics, and generalising to instances larger than those seen during training [31]. This work established bipartite GNNs as the dominant neural architecture for branching.

For industrial-scale instances where even GPU-accelerated SB would be too slow, Nair et al. [32] propose Neural Branching with a GPU-scaled SB labelling pipeline. The resulting models, when integrated into SCIP, improve the primal–dual gap by 2×–10× on most evaluated datasets and by extreme margins on one production dataset, demonstrating that the SB imitation paradigm scales to real-world problem sizes [32]. At the opposite extreme of model complexity, sparse linear models that approximate SB scores with far fewer parameters than GNNs can match GNN accuracy and even outperform GPU GNNs in CPU-only deployment, offering a practical middle ground between expressivity and deployment cost [33].

5.1.2 Graph Neural Network Branching Policies

The bipartite GNN paradigm of Gasse et al. [31] has been extended in several directions. The graph pointer network of [12] combines a GNN backbone with a pointer mechanism, incorporating global and historical solver state features alongside graph features; the model is trained with a top-k Kullback–Leibler divergence loss and demonstrates superior performance over expert-designed rules on set covering, facility location, maximum independent set, and graph coloring benchmarks [12]. Du et al. [34] propose to learn both variable selection and node selection simultaneously within a single bipartite GNN, reporting faster solving times and smaller trees than SCIP on multiple datasets [34].

GNN-based policies enjoy permutation invariance with respect to variable and constraint reordering, enabling generalisation across structurally similar but syntactically distinct instances. However, they impose non-trivial training and inference costs, require costly SB labels for supervision, and their effectiveness degrades when the test-time instance distribution diverges from the training corpus [31] [33].

5.1.3 Reinforcement Learning for Branching

Reinforcement learning (RL) offers an alternative training signal that bypasses SB label collection by directly optimising end-to-end solver metrics—node count, LP iterations, or wall-clock time—through interaction with a solver environment.

Etheve et al. [35] design a tailored RL architecture with local value functions whose structure is consistent with global solver objectives, demonstrating that learned RL policies can generalise across instances and compete with commercial solver branching strategies [35]. Alès et al. [36] provide a principled MDP formalisation of the B&B variable-selection problem (BBMDP) that enables standard RL algorithms to be applied without ad hoc modifications; agents trained under this formulation outperform prior RL agents on four standard MILP benchmarks [36].

A persistent obstacle is the sparse, terminal nature of B&B rewards: a branching decision at a node early in the tree affects the entire subtree explored subsequently, yet the reward signal (e.g., total node count) is only observed at termination. Dou et al. [37] address this with ReviBranch, which constructs *revived trajectories* that link each branching decision to the evolving graph states of the subtree it influences, combined with reward redistribution to densify the training signal. ReviBranch reduces B&B nodes by 4.0% and LP iterations by 2.2% on large instances versus prior RL baselines [37]. While these absolute improvements are modest, they point toward a principled framework for the credit-assignment challenge that underlies most RL-for-branching approaches.

5.1.4 Online and Adaptive Branching

Rather than fixing a policy after offline training, online and adaptive methods update branching strategies during solving or across a recurring stream of similar instances, exploiting distribution-specific structure that a generic pre-trained model cannot capture.

The on-the-fly learning framework of Khalil et al. [30] is the canonical instance-adaptive approach. Alès et al. [38] extend online learning to a graph-oriented setting: a Thompson sampling strategy ranks variable representations computed from the bipartite graph at each node, yielding results competitive with state-of-the-art online baselines and generalising across instances with varying coefficients [38]. For recurring operational settings—where the same problem class is solved repeatedly—the authors of [39] propose PreMIO, a pre-trained multi-variable cardinality branching approach with theoretically grounded hyperparameter selection that is validated on both standard OR benchmarks and real-world instances [39]. Exploiting similarity at a finer granularity, the authors of [40] train separate models per variable group to leverage recurring instance structure, reporting improved gap closure on realistic unit commitment instances compared to single-model baselines using the same training budget [40].

5.1.5 Node Selection, Diving, and Primal Heuristics

Beyond branching, ML has been applied to node selection and to primal heuristics that guide the B&B search toward feasible solutions.

Nair et al. [32] complement Neural Branching with Neural Diving: a neural network produces a partial variable assignment (a dive), which is then completed by the solver's internal repair mechanisms. Combining both components in SCIP yields the largest reported improvements, with primal–dual gap reductions of 2×–10× on most datasets and substantially larger gains on one production dataset [32].

An alternative primal strategy exploits *backdoors*—small subsets of variables whose branching suffices to propagate solutions efficiently. Ferber et al. [41] train a model to predict pseudo-backdoors for MIPs via supervised learning, demonstrating improved solve performance on generalised independent set problems [41]. Cai et al. [42] extend this paradigm by using Monte Carlo tree search (MCTS) to collect high-quality backdoor candidates and training a graph attention network with a contrastive loss to predict them; the resulting model outperforms both prior backdoor models and Gurobi across several MILP benchmark domains [42].

Machine learning augmented branch-and-bound thus encompasses a rich hierarchy of decisions—from variable selection and node ordering to primal heuristics—and extends beyond branching decisions to other algorithmic components [43]. By learning multiple aspects of the branch-and-bound process, these approaches can achieve substantial speedups while maintaining solution quality guarantees.

5.2 Constraint Programming Integration

Constraint programming provides a declarative framework for modeling and solving combinatorial problems. Integrating machine learning with constraint programming can enhance solver performance through learned heuristics.

A general hybrid approach combines deep reinforcement learning with constraint programming via a dynamic programming formulation that bridges both techniques [10]. This method outperforms standalone RL and constraint programming solutions while remaining competitive with industrial solvers on the TSP with time windows and portfolio optimization problems [10].

For job shop scheduling, supervised learning can automatically learn variable ordering strategies for constraint programming solvers [14]. Two models are developed: a regression model predicting optimal start times and a classification model predicting optimal operation sequences. Using support vector machines, multi-layer perceptrons, and genetic programming, the approach demonstrates competitive performance against state-of-the-art methods. Hybridization of ML-based and domain-based variable ordering methods further improves solver performance [14].

Deep Q-learning can be trained directly inside a generic constraint programming solver to learn value-selection heuristics [15]. A restart-based training procedure defines episodes as single dives in the search tree without backtracks, while a propagation-based reward function assigns intermediate rewards based on domain reductions. Using a heterogeneous GNN architecture with tripartite graph representation, this approach learns effective heuristics for graph coloring, maximum independent set, and maximum cut [15].

5.3 Column Generation Enhancement

Column generation is a powerful technique for solving large-scale optimization problems by iteratively generating promising variables. Machine learning can accelerate the pricing subproblem, which is often the computational bottleneck.

A transformer pointer network has been applied to predict optimal solutions for single machine scheduling subproblems in column generation [21]. The supervised learning framework

trains the model offline via teacher-forcing, and the neural network predicts candidate job sequences instead of using dynamic programming. Optimality guarantees are preserved through exact dynamic programming verification at termination. The approach demonstrates generalization across varying instance sizes and outperforms traditional column generation with dynamic programming [21].

For joint rider trip planning and crew shift scheduling, AGGNNI-CG hybridizes column generation with a GNN to accelerate pricing subproblems [11]. The supervised GNN features multi-head attention and residual gated graph architecture, predicting promising edges to reduce search space. Evaluation on real-world paratransit datasets shows significant improvements over traditional column generation [11].

5.4 Mixed-Integer Programming Acceleration

For recurrent mixed-integer programs, online learning approaches can discover good solutions efficiently. An entropy minimization heuristic uses an online Fast Random Forest model updated dynamically during branch-and-bound [44]. The model uses feature vectors based on statistics of visited solutions and employs entropy to measure variable stability and guide lazy constraint addition. This approach avoids deep learning's high data and tuning overheads while achieving an order of magnitude speedup over CPLEX on the locomotive assignment problem with relative gaps below 2% [44].

Parametric mixed-integer optimization can be accelerated by replacing solvers with neural network prediction [45]. Feedforward neural networks perform multiclass classification to map parameters to optimal strategies, where strategies encode tight constraints and integer variable values. Strategy pruning via frequency-based heuristics reduces strategy counts while maintaining feasibility. This approach achieves up to three orders of magnitude speedups over Gurobi on real-world benchmarks including fuel cell energy management, portfolio trading, and motion planning [45].

5.5 Learning Problem Approximations

Rather than solving problems directly, machine learning can learn to approximate hard problems by easier variants. A structured learning approach based on maximum conditional likelihood uses feature-based ML predictors to transform hard problem instances into easy ones [19]. This paradigm provides algorithms to approximate arbitrary path problems on acyclic digraphs and path partition problems via flow problems. A column generation matheuristic demonstrates efficiency on stochastic vehicle scheduling problems [19].

Learning objective boundaries for constraint optimization problems enables search space pruning [18]. The Bion approach uses supervised machine learning to learn from previously solved instances, accurately under- and overestimating the optimum variable. Experimental evaluation across seven realistic constraint optimization problems shows that the estimation model prunes objective variable domains by over 80%, improving solving efficiency for some problems [18].

6 Problem Domains and Applications

Machine learning approaches have been applied across a diverse range of combinatorial optimization problem domains. This section surveys the major application areas and problem-specific innovations.

6.1 Routing Problems

Routing problems, particularly the Traveling Salesman Problem and Vehicle Routing Problem variants, have been the most extensively studied application domain for neural combinatorial optimization. The TSP serves as a canonical benchmark problem, appearing in numerous studies [5], [6], [10], [12], [46], [25]. The capacitated vehicle routing problem has been addressed through efficient active search strategies that surpass the LKH3 heuristic solver [4].

Multi-objective routing problems present additional challenges. Pareto set learning enables simultaneous optimization of multiple objectives in multi-objective TSP and vehicle routing problems [23]. The Covering Salesman Problem, which requires visiting a subset of cities such that all cities are covered within a specified radius, has been solved using deep RL with dynamic embeddings that capture changing covering states [6].

Specialized routing applications include joint rider trip planning and crew shift scheduling for paratransit services [11]. The AGGNNI-CG approach combines column generation with GNNs to handle the complex constraints of coordinating rider trips with crew schedules, demonstrating practical applicability on real-world datasets from Chatham County, Georgia [11].

6.2 Scheduling Problems

Scheduling problems encompass a wide variety of resource allocation and sequencing challenges. Job shop scheduling has been addressed through supervised learning of variable ordering strategies for constraint programming solvers [14]. Regression and classification models predict optimal start times and operation sequences, with hybridization of ML-based and domain-based methods improving performance [14].

Parallel machine scheduling problems have been tackled using transformer pointer networks integrated with column generation [21]. The approach predicts job sequences for single machine subproblems, generalizing across varying instance sizes while maintaining optimality guarantees [21]. Diffusion models have also achieved state-of-the-art performance on parallel machine scheduling with unrelated machines [24].

The one-dimensional cutting stock problem, which involves cutting raw materials to minimize waste, has been formulated as a sequential decision problem and solved using deep RL with pointer networks [20]. The approach achieves high-efficiency solutions in under one second per instance [20].

6.3 Graph Optimization Problems

Graph optimization problems naturally align with graph neural network architectures. Maximum cut, maximum independent set, and minimum vertex cover have been extensively studied using physics-inspired GNN approaches [8], [22], [15]. These problems can be formulated as energy minimization tasks with differentiable loss functions, enabling unsupervised training [8].

Maximum clique enumeration benefits from interpretable ML frameworks that use local features to prune non-optimal nodes [17]. Multi-stage sparsification can eliminate 99% of nodes in sparse graphs while retaining maximum cliques, and novel heuristics like ALTHEA emerge from learned features [17].

Graph coloring presents unique challenges due to its constraint satisfaction nature. Deep Q-learning trained inside constraint programming solvers has demonstrated effectiveness on graph coloring alongside maximum independent set and maximum cut [15].

Dominating set variants have been addressed through neural network frameworks that frame the problem as constraint satisfaction [28]. The multi-hop influence maximization problem, formulated as a k-d-dominating set problem, has been solved by integrating large language models with metaheuristics [29].

6.4 Mixed-Integer Programming

Mixed-integer programming encompasses a broad class of optimization problems with both continuous and discrete variables. Learning to branch has been extensively studied for MIP solvers [12], [43]. Graph pointer networks learn branching rules that outperform expert-designed heuristics across diverse MIP instances [12].

For recurrent MIPs that share similar structure across instances, online learning approaches can exploit this structure. Entropy minimization with Fast Random Forests achieves order-of-magnitude speedups on the locomotive assignment problem [44]. Parametric MIP problems can be solved in milliseconds by learning to map parameters to optimal strategies, achieving three orders of magnitude speedups on fuel cell energy management and portfolio trading applications [45].

Bilevel optimization problems, which involve hierarchical decision-making, have been addressed using GNNs that model the problem as a tripartite graph [26]. The approach transforms the bilevel knapsack problem into a single-level problem, achieving 500-fold speedups with minimal optimality gaps [26].

6.5 Constraint Optimization Problems

Constraint optimization problems combine objective optimization with constraint satisfaction. Learning objective boundaries enables search space pruning by estimating upper and lower bounds on the optimal objective value [18]. The Bion approach demonstrates over 80% domain reduction across seven realistic constraint optimization problems [18].

Stochastic and chance-constrained optimization problems introduce uncertainty into problem formulations. Learning to approximate hard stochastic vehicle scheduling problems by easier deterministic variants enables efficient solution through structured learning and column generation matheuristics [19].

6.6 Emerging Applications

Beyond traditional problem domains, machine learning for combinatorial optimization is being applied to emerging applications. Virtual network function placement in telecommunications has been addressed using deep reinforcement learning [47]. Bayesian optimization of combinatorial structures provides adaptive, scalable models for problems with scarce data [48].

The FrontierCO benchmark evaluates neural solvers on real-world, large-scale combinatorial optimization problems, comparing performance against traditional optimization methods and constraint programming approaches [49]. Such benchmarking efforts are crucial for assessing the practical applicability of ML methods beyond academic problem instances.

7 Discussion

7.1 Performance and Scalability

The literature reveals significant progress in the performance of machine learning methods for combinatorial optimization. Several studies report competitive or superior performance compared to classical heuristics. Efficient active search strategies surpass the well-known LKH3 solver on capacitated vehicle routing problems [4]. Physics-inspired GNN methods perform on par with or outperform existing solvers while scaling to problems with millions of variables [8]. Hybrid approaches combining learning with column generation or branch-and-bound achieve substantial speedups, with some methods reporting order-of-magnitude improvements [44], [45].

However, performance comparisons must be interpreted carefully. Many studies evaluate on relatively small problem instances or specific problem distributions that may not reflect real-world complexity. The RL4CO benchmark reveals that some recent state-of-the-art methods

fall behind their predecessors when evaluated on metrics beyond single-instance performance, highlighting the importance of comprehensive evaluation [25]. Generalization to larger instances remains challenging, though some approaches demonstrate the ability to solve instances much larger than those seen during training [4], [5].

Scalability varies significantly across methods. Autoregressive construction methods face sequential bottlenecks that limit parallelization, though approaches like DIMES address this through continuous parameterization of solution distributions [5]. GNN methods generally scale well due to their local message-passing operations, with some achieving linear time complexity [7]. Hybrid methods inherit the scalability characteristics of the classical optimization components they enhance.

7.2 Training Requirements and Data Efficiency

A critical consideration for practical deployment is the computational cost and data requirements of training. Supervised learning approaches require labeled training data, typically optimal or high-quality solutions obtained from exact solvers [14], [12]. This creates a chicken-and-egg problem: if we could efficiently solve instances to generate training data, we might not need the learned model. Consequently, supervised methods are often limited to smaller training instances, raising questions about generalization to larger or structurally different test instances.

Reinforcement learning methods avoid the need for optimal solutions, requiring only a reward signal based on solution quality [5], [6], [22]. However, RL training can be sample-inefficient and computationally expensive, often requiring millions of training episodes. Meta-learning approaches aim to improve sample efficiency by learning across problem distributions [5].

Unsupervised methods offer an appealing alternative by training directly on the problem's objective function without labeled data [7], [8]. Physics-inspired GNN approaches eliminate training data generation entirely, though they may require careful tuning of loss functions and training procedures.

Online learning approaches that adapt to specific instances at test time present another paradigm [4], [44]. Efficient active search updates only a subset of model parameters during test-time adaptation, balancing adaptation benefits with computational costs [4].

7.3 Solution Quality and Optimality Guarantees

A fundamental limitation of most machine learning approaches is the lack of optimality guarantees. While classical exact methods guarantee optimal solutions (given sufficient time), and approximation algorithms provide bounded approximation ratios, learned heuristics typically offer no such guarantees. Solution quality depends on training data, model capacity, and the specific problem instance.

Hybrid approaches that integrate learning with exact methods can preserve optimality guarantees. For example, learning-enhanced column generation maintains optimality by using exact dynamic programming verification at termination [21]. Similarly, learning to branch in branch-and-bound preserves the exactness of the underlying algorithm while potentially improving efficiency [12].

The trade-off between solution quality and computational time is central to evaluating ML methods. Many approaches achieve near-optimal solutions orders of magnitude faster than exact methods [44], [26], [45]. For large-scale problems where exact methods are intractable, learned heuristics may provide the only feasible approach.

7.4 Interpretability and Insights

Most deep learning approaches for combinatorial optimization operate as black boxes, offering limited insight into why particular solutions are generated. This lack of interpretability can

hinder trust and adoption in critical applications. Some work addresses this through interpretable ML frameworks that use intuitive local features [17]. The discovery of novel heuristics like ALTHEA from learned features demonstrates that ML can provide problem insights beyond pure solution generation [17].

Hybrid approaches that enhance specific components of classical algorithms may offer better interpretability, as the overall algorithmic structure remains transparent even if individual learned components are opaque. Learning objective boundaries or search space pruning strategies can provide interpretable guidance to traditional solvers [17], [18].

7.5 Generalization and Robustness

Generalization across problem instances is crucial for practical applicability. Several studies demonstrate generalization to instances larger than training instances [4], [5], [21]. However, generalization to structurally different problem distributions remains challenging. Active adaptation methods that fine-tune models on out-of-distribution instances show promise for improving robustness [23].

The ability to handle problem variants and constraints is another aspect of generalization. Some approaches demonstrate flexibility across multiple problem types within a domain, such as different vehicle routing variants [25]. However, most methods are problem-specific and require retraining for new problem classes.

7.6 Computational Infrastructure Requirements

Training deep learning models for combinatorial optimization often requires substantial computational resources, including GPU clusters and extended training times. This creates barriers to entry for researchers and practitioners without access to such infrastructure. Inference costs are generally lower but still significant for large-scale problems or real-time applications.

Some approaches explicitly address computational efficiency. Online mixed-integer optimization achieves millisecond solve times by replacing solvers with neural network prediction [45]. Efficient active search reduces test-time adaptation costs by updating only parameter subsets [4]. The trade-off between training cost and inference efficiency must be considered in the context of specific applications.

7.7 Benchmarking and Reproducibility

Standardized benchmarking is essential for comparing methods and tracking progress. The RL4CO benchmark provides a unified framework for evaluating reinforcement learning approaches [25]. The ML4CO competition has catalyzed research by providing standardized problem instances and evaluation protocols [50]. FrontierCO focuses on real-world, large-scale evaluation [49].

However, reproducibility remains a challenge. Many papers lack sufficient implementation details or publicly available code. Differences in problem formulations, instance distributions, and evaluation metrics complicate direct comparisons across studies. The development of standardized libraries and benchmarks is crucial for advancing the field [25], [51].

8 Future Directions and Open Challenges

8.1 Unified Frameworks and Architectures

The field would benefit from unified frameworks that can handle multiple problem types with minimal problem-specific customization. While current approaches are largely problem-specific,

some work explores more general architectures [9], [25]. Developing universal neural combinatorial optimization solvers that can adapt to diverse problem structures through meta-learning or transfer learning represents an important research direction.

The integration of different ML paradigms also holds promise. Combining the strengths of supervised learning, reinforcement learning, and unsupervised learning in a unified framework could leverage complementary advantages. For example, unsupervised pre-training followed by supervised or RL fine-tuning might improve sample efficiency and generalization.

8.2 Theoretical Understanding

Despite empirical successes, theoretical understanding of why and when machine learning methods work for combinatorial optimization remains limited. Developing approximation guarantees, sample complexity bounds, and generalization theory for learned heuristics would strengthen the field's foundations. Understanding the relationship between problem structure, neural architecture, and solution quality could guide more principled model design.

The connection between optimization landscapes and neural network training dynamics deserves deeper investigation. Why do certain architectures succeed on specific problem classes? What problem characteristics make learning-based approaches more or less effective than classical methods? Addressing these questions could lead to more robust and predictable methods.

8.3 Large Language Models and Foundation Models

The emergence of large language models and foundation models opens new possibilities for combinatorial optimization. Initial work integrating LLMs with metaheuristics demonstrates improved performance on influence maximization problems [29]. LLMs' ability to understand problem descriptions, generate code, and reason about problem structure could enable more flexible and generalizable optimization systems.

Foundation models pre-trained on diverse optimization problems could provide strong initialization for fine-tuning on specific problem instances or classes. The few-shot learning capabilities of large models might enable rapid adaptation to new problem variants with minimal training data. However, the computational costs and interpretability challenges of large models must be carefully considered.

8.4 Hybrid and Neuro-Symbolic Approaches

Further development of hybrid approaches that tightly integrate learning with classical optimization techniques represents a promising direction [16]. Rather than replacing traditional solvers, machine learning can enhance specific components while preserving desirable properties like optimality guarantees and interpretability. Learning multiple components of optimization algorithms simultaneously, rather than individual components in isolation, could yield synergistic improvements.

Neuro-symbolic approaches that combine neural networks with symbolic reasoning and constraint propagation could leverage both data-driven learning and logical inference. Such methods might achieve better generalization and interpretability while maintaining the flexibility of learned models.

8.5 Real-World Deployment and Applications

Transitioning from academic benchmarks to real-world deployment requires addressing several challenges. Real-world problems often involve complex constraints, uncertain parameters, and dynamic environments that are not well-represented in standard benchmarks. Developing methods that handle uncertainty, incorporate domain knowledge, and adapt to changing conditions is crucial for practical impact.

The computational infrastructure requirements for training and deploying learned models must be reduced for widespread adoption. Efficient architectures, knowledge distillation, and edge deployment strategies could make ML-based optimization more accessible. Developing user-friendly tools and interfaces that enable domain experts without ML expertise to apply these methods would accelerate adoption.

8.6 Multi-Task and Transfer Learning

Learning optimization strategies that transfer across related problems could dramatically improve sample efficiency and generalization. Multi-task learning approaches that train on multiple problem types simultaneously might discover shared structure and develop more robust representations. Transfer learning from well-studied problems to novel or data-scarce problems could enable rapid deployment in new domains.

Meta-learning approaches that learn to learn optimization strategies show promise for few-shot adaptation to new problem instances or classes [5]. Developing meta-learning methods that can handle significant distribution shifts and structural differences between training and test problems remains an open challenge.

8.7 Explainability and Trust

For adoption in critical applications, machine learning methods must provide explanations for their decisions and inspire trust in their solutions. Developing interpretable neural architectures, attention visualization techniques, and post-hoc explanation methods tailored to combinatorial optimization could improve transparency. Certifying solution quality or providing confidence estimates would help users assess when to trust learned solutions versus invoking more expensive exact methods.

Interactive optimization systems that combine human expertise with learned models could leverage the strengths of both. Allowing domain experts to guide or constrain the learning process, inspect intermediate solutions, and override learned decisions when necessary could improve both solution quality and user acceptance.

8.8 Benchmarking and Evaluation Standards

The field needs more comprehensive and standardized benchmarking practices. Beyond solution quality on single instances, evaluation should consider generalization across instance distributions, scalability to larger problems, robustness to problem variations, sample efficiency, training costs, and inference time. Developing benchmark suites that include real-world problem instances with practical constraints would better assess methods' practical applicability [49].

Reproducibility standards including public code repositories, detailed hyperparameter specifications, and standardized evaluation protocols would accelerate progress. Community-driven benchmarking competitions like ML4CO [50] play an important role in establishing standards and driving innovation.

8.9 Quantum and Neuromorphic Computing

Emerging computing paradigms offer potential advantages for combinatorial optimization. Quantum computing algorithms for optimization problems could be enhanced through machine learning for problem encoding, parameter tuning, or hybrid classical-quantum approaches. Neuromorphic computing hardware designed to efficiently execute neural network computations could reduce the energy and time costs of learned optimization methods.

The intersection of machine learning, combinatorial optimization, and novel computing architectures represents an exciting frontier that could enable solving previously intractable problems at scale.

9 Conclusion

Machine learning has emerged as a transformative approach to combinatorial optimization, offering data-driven methods that can learn problem-specific heuristics, exploit structural patterns, and scale to large problem instances. This literature review has examined 30 highly relevant papers spanning three primary methodological paradigms: deep reinforcement learning approaches that learn constructive or improvement heuristics, graph neural network methods that leverage problem structure through unsupervised or physics-inspired training, and hybrid frameworks that integrate learning with classical optimization techniques.

The field has achieved significant milestones. Deep RL methods have demonstrated the ability to construct high-quality solutions for routing, scheduling, and graph optimization problems, with some approaches surpassing classical heuristics like LKH3 [4]. Graph neural networks have scaled to problems with millions of variables while maintaining competitive solution quality [8]. Hybrid approaches have achieved order-of-magnitude speedups by enhancing specific components of traditional solvers while preserving optimality guarantees [44], [21], [45].

However, substantial challenges remain. Most methods lack optimality guarantees and struggle with generalization to problem distributions significantly different from training data. Training requirements can be substantial, requiring large amounts of labeled data for supervised methods or extensive sampling for reinforcement learning. Interpretability remains limited for most deep learning approaches, hindering trust and adoption in critical applications. Standardized benchmarking and reproducibility practices need further development to enable fair comparisons and track progress.

The integration of machine learning with combinatorial optimization represents a paradigm shift from manually designed heuristics to learned, data-driven approaches. Rather than replacing classical optimization entirely, the most promising direction appears to be hybrid methods that combine the strengths of both paradigms. Machine learning can discover patterns and heuristics from data, while classical optimization provides structure, guarantees, and interpretability.

Looking forward, several directions hold particular promise. Unified frameworks that handle multiple problem types with minimal customization could democratize access to neural combinatorial optimization. Integration with large language models and foundation models might enable more flexible and generalizable systems. Tighter integration of learning with classical optimization techniques could preserve desirable properties while achieving learning-based improvements. Real-world deployment requires addressing uncertainty, dynamic environments, and complex constraints beyond standard benchmarks.

The field of machine learning for combinatorial optimization is rapidly evolving, with new architectures, training paradigms, and applications emerging regularly. As methods mature and transition from academic benchmarks to real-world deployment, they have the potential to transform how we approach optimization problems across science, engineering, and industry. The synthesis of data-driven learning with decades of optimization research offers a powerful toolkit for tackling the combinatorial challenges of the 21st century.

References

[1] D. Bertsimas and B. Stellato, "The voice of optimization," 2021.

[2] M. S. A. Sakhri, A. Goëffon, O. Goudet, F. Saubion, and C. Touhami, "Discovering new robust local search algorithms with neuro-evolution," 2025.

[3] J. Kotary, F. Fioretto, P. V. Hentenryck, and B. Wilder, "End-to-end constrained optimization learning: A survey," 2021.

[4] A. Hottung, Y.-D. Kwon, and K. Tierney, "Efficient active search for combinatorial optimization problems.," 2021.

[5] R. Qiu, Z. Sun, and Y. Yang, "Dimes: A differentiable meta solver for combinatorial optimization problems," 2022.

[6] "Deep reinforcement learning for combinatorial optimization: Covering salesman problems," 2022.

[7] T. Peng, K. Aihara, and L. Chen, "Brain-inspired chaotic graph backpropagation for large-scale combinatorial optimization," 2024.

[8] "Combinatorial optimization with physics-inspired graph neural networks," 2022.

[9] H.-J. Hung, W.-C. Lee, T.-Y. Fu, C.-Y. Shen, and Z. Lei, "Learning to solve combinatorial optimization problems on graphs with state-aware multi-relation aggregation," 2024.

[10] Q. Cappart, T. Moisan, L.-M. Rousseau, I. Prémont-Schwarz, and A. A. Cire, "Combining reinforcement learning and constraint programming for combinatorial optimization," 2020.

[11] J. Lu, T. Ye, W. Chen, and P. V. Hentenryck, "Boosting column generation with graph neural networks for joint rider trip planning and crew shift scheduling," 2024.

[12] "Learning to branch in combinatorial optimization with graph pointer networks," 2023.

[13] A. Deepak, D. Bistra, and G. Tias, "Machine learning augmented algorithms for combinatorial optimization problems (dagstuhl seminar 24441)," 2025.

[14] "Enhancing constraint programming via supervised learning for job shop scheduling," 2022.

[15] "Training a deep q-learning agent inside a generic constraint programming solver," 2023.

[16] A. Lallouet, "New ai techniques for combinatorial optimization," 2022.

[17] J. Lauri, S. Dutta, M. Grassia, and D. Ajwani, "Learning fine-grained search space pruning and heuristics for combinatorial optimization.," 2020.

[18] M. Gasse, Q. Cappart, J. Charfreitag, L. Charlin, D. Ch'etelat, A. Chmiela, J. Dumouchelle, A. M. Gleixner, A. M. Kazachkov, E. B. Khalil, P. Lichocki, A. Lodi, M. Lubin, C. J. Maddison, C. Morris, D. J. Papageorgiou, A. Parjadis, S. Pokutta, A. Prouvost, L. Scavuzzo, G. Zarpellon, L. Yangm, S.-J. Lai, A. Wang, X. Luo, X. Zhou, H. Huang, S. C. Shao, Y.-L. Zhu, D. D. Zhang, T. M. Quan, Z. Cao, Y. Xu, Z. Huang, S. Zhou, C. Binbin, H. Minggui, H. Hao, Z. Zhiyu, A. Zhiwu, and M. Kun, "The machine learning for combinatorial optimization competition (ml4co): Results and insights," 2022.

[19] "Learning to approximate industrial problems by operations research classic problems," 2022.

[20] J. Fang, Y. Rao, Q. Luo, and J. Xu, "Solving one-dimensional cutting stock problems with the deep reinforcement learning," 2023.

[21] A. Hijazi, O. Y. Özaltın, and R. Uzsoy, "All you need is an improving column: Enhancing column generation for parallel machine scheduling via transformers," 2024.

[22] T. D. Barrett, W. R. Clements, J. Foerster, and A. I. Lvovsky, "Exploratory combinatorial optimization with reinforcement learning," 2019.

[23] "Pareto set learning for neural multi-objective combinatorial optimization," 2022.

[24] S.-H. Hong, H. Kim, Z. Jang, and B.-J. Lee, "Ic/dc: Surpassing heuristic solvers in combinatorial optimization with diffusion models," 2024.

[25] H. Spieker and A. Gotlieb, "Learning objective boundaries for constraint optimization problems." arXiv, 2020.

[26] "Solving bilevel knapsack problem using graph neural networks," 2022.

[27] S. K. Ali and D. Boughaci, "Combining simulated annealing with logistic regression for binary combinatorial optimization problems," 2023.

[28] R. Baptista and M. Poloczek, "Bayesian optimization of combinatorial structures." arXiv, 2018.

[29] C. C. Sartori, C. Blum, F. Bistaffa, and G. R. Corominas, "Metaheuristics and large language models join forces: Towards an integrated optimization approach," 2024.

[30] E. B. Khalil, P. L. Bodic, L. Song, G. L. Nemhauser, and B. Dilkina, "Learning to branch in mixed integer programming," 2016. AAAI Conference on Artificial Intelligence, Vol. 30, No. 1, pp. 724–731.

[31] M. Gasse, D. Chételat, N. Ferroni, L. Charlin, and A. Lodi, "Exact combinatorial optimization with graph convolutional neural networks," 2019. Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, pp. 15554–15566.

[32] V. Nair, S. Bartunov, F. Gimeno, I. von Glehn, P. Lichocki, I. Lobov, B. O'Donoghue, N. Sonnerat, C. Tjandraatmadja, P. Wang, and R. Addanki, "Solving mixed integer programs using neural networks," 2020. arXiv preprint arXiv:2012.13349.

[33] S. Bayramoglu, G. L. Nemhauser, and N. V. Sahinidis, "Speeding up mixed-integer programming solvers with sparse learning for branching," 2026. arXiv preprint arXiv:2604.00094.

[34] S. Du, J. Tong, and W. Fan, "Learning efficient branch-and-bound for solving mixed integer linear programs," 2024. SSRN preprint, doi:10.2139/ssrn.4822233.

[35] M. Etheve, Z. Alès, C. Bissuel, O. Juan, and S. Kedad-Sidhoum, "Reinforcement learning for variable selection in a branch and bound algorithm," in *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*, pp. 176–185, 2020.

[36] Z. Alès, O. Juan, S. Kedad-Sidhoum, and E. Rachelson, "A markov decision process for variable selection in branch & bound," 2025. arXiv preprint arXiv:2510.19348.

[37] J. Dou, J. Nie, Y. Cheng, J. Liu, Y. Ji, C. Xiao, F. Du, and J. Xiao, "Revibranch: Deep reinforcement learning for branch-and-bound with revived trajectories," 2025. arXiv preprint arXiv:2508.17452.

[38] Z. Alès, O. Juan, S. Kedad-Sidhoum, and E. Rachelson, "Influence branching for learning to solve mixed-integer programs online," 2025. arXiv preprint arXiv:2510.04273.

[39] "Pre-trained mixed integer optimization through multi-variable cardinality branching," 2023. arXiv preprint arXiv:2305.12352.

[40] "Exploiting instance and variable similarity to improve learning-enhanced branching," 2022. arXiv preprint arXiv:2208.10028.

[41] A. Ferber, J. Song, B. Dilkina, and Y. Yue, "Learning pseudo-backdoors for mixed integer programs," 2021. arXiv preprint arXiv:2106.05080.

[42] J. Cai, T. Huang, and B. Dilkina, "Learning backdoors for mixed integer linear programs with contrastive learning," 2024. arXiv preprint arXiv:2401.10467.

[43] Q. Wang, "Alpha-t: Learning to traverse over graphs with an alphazero-inspired self-play framework," 2021.

[44] "Minimizing entropy to discover good solutions to recurrent mixed integer programs," 2022.

[45] F. Berto, C. Hua, J.-Y. Park, M. Kim, H.-S. Kim, J. Son, H. Kim, J. Kim, and J. Park, "Rl4co: an extensive reinforcement learning for combinatorial optimization benchmark," 2023.

[46] J. Yan, S. Yang, and E. R. Hancock, "Learning for graph matching and related combinatorial optimization problems," 2020.

[47] M. Lombardi and M. Milano, "Boosting combinatorial problem modeling with machine learning," 2018.

[48] "Online mixed-integer optimization in milliseconds," 2022.

[49] H. Liu, Y. Kuang, J. Wang, X. Li, Y. Zhang, and F. Wu, "Promoting generalization for exact solvers via adversarial instance augmentation," 2023.

[50] M. Seyfi, A. Banitalebi-Dehkordi, Y. Shi, and Y. Zhang, "Exact combinatorial optimization with temporo-attentional graph neural networks," 2023.

[51] "Learning computation bounds for branch-and-bound algorithms to k-plex extraction," 2022.